M. Siri Chandana

2405A41139

1.Two Sum LC-1

https://leetcode.com/problems/two-sum/submissions/2186383545/

Description | Editorial | Solutions | Submissions

1. Two Sum

Solved

Easy | Topics | Companies | Hint

You are given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15], target = 9`

Output: `[0,1]`

Explanation:

The sum of `nums[0]` and `nums[1]` is `2 + 7 = 9`, which equals the target. Therefore, the indices `[0, 1]` are returned.

Example 2:

Input: `nums = [3,2,4], target = 6`

71K | 2.2K | 810 Online

</> Code

Java | Auto

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
    int a = -1;
    int b = -1;
    for(int i=0;i<nums.length;i++){
        for(int j=i+1;j<nums.length;j++){
            if(nums[i] + nums[j] == target){
                a = i;
                b = j;
                break;
            }
```

Saved | Ln 1, Col 1

Testcase | Test Result

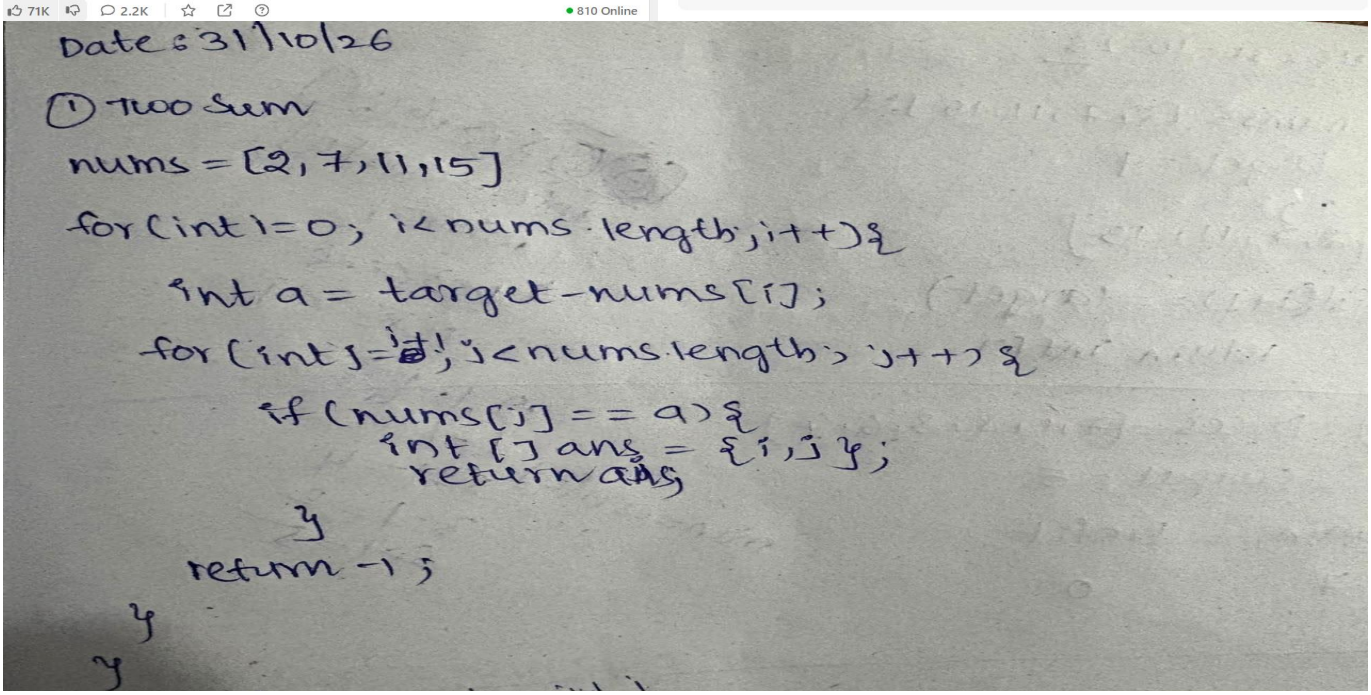
Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[2,7,11,15]

target =
9



2.Best Time to Buy and Sell Stock LC-121

https://leetcode.com/problems/best-time-to-buy-and-sell-stock/submissions/2186387135/

Description | Accepted | Editorial | Solutions | Submissions

121. Best Time to Buy and Sell Stock

Solved

Easy | Topics | Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the ith day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6),
profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed
because you must buy before you sell.
```

Example 2:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max
profit = 0.
```

Constraints:

- `1 <= prices.length <= 10^5`
- `0 <= prices[i] <= 10^4`

36.6K | 887

</> Code

Java | Auto

```java
class Solution {
    public int maxProfit(int[] prices) {

        int min = prices[0];
        int profit = 0;

        for(int i = 1; i < prices.length; i++) {

            if(prices[i] < min) {
                min = prices[i];
            }

            else if(prices[i] - min > profit) {
                profit = prices[i] - min;
            }
        }

        return profit;
    }
}
```

Saved | Ln 16, Col 10

Testcase | >_ Test Result

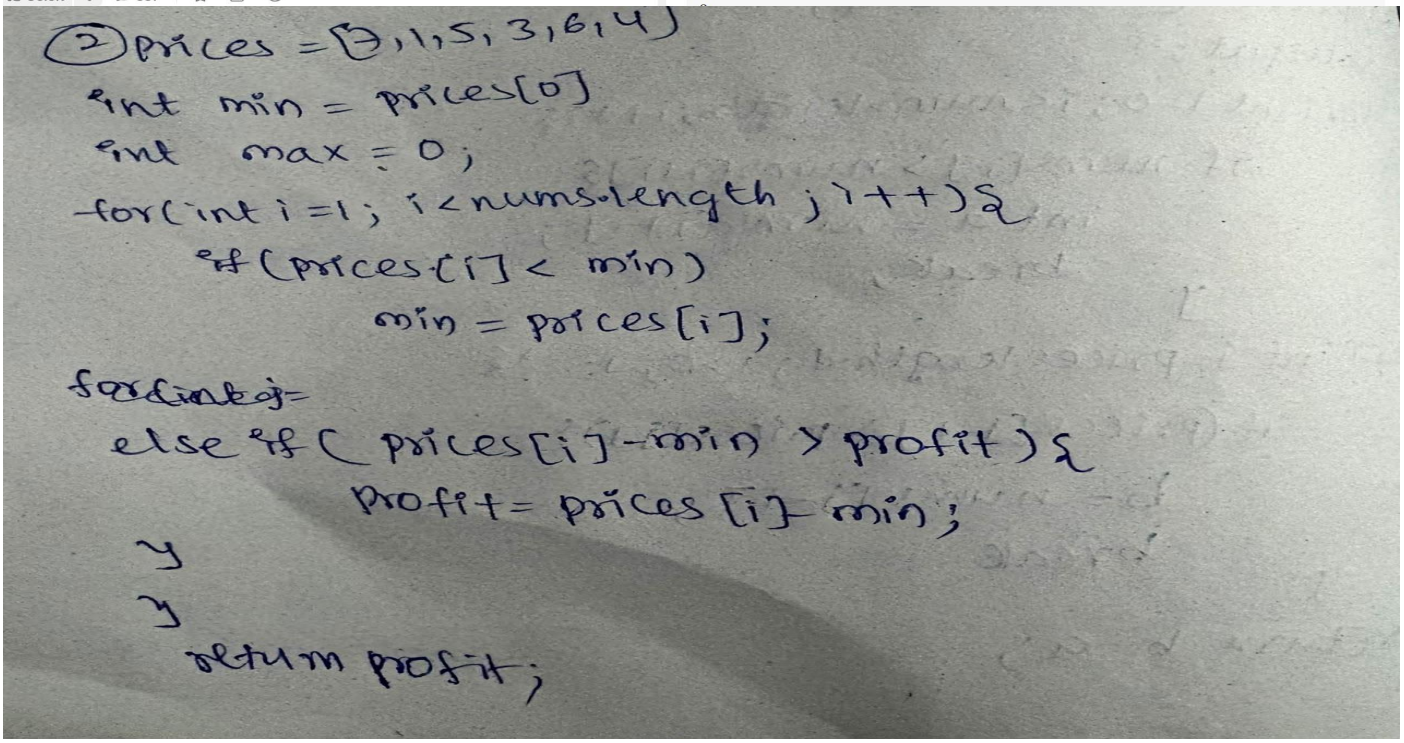
Accepted Runtime: 0 ms

Case 1 | Case 2

Input

prices =
[7,6,4,3,1]

Output



3. Move Zeros LC-283

https://leetcode.com/problems/move-zeroes/submissions/2186416338/

283. Move Zeroes

Solved

Easy | Topics | Companies | Hint

Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

Follow up: Could you minimize the total number of operations done?

Java | Auto

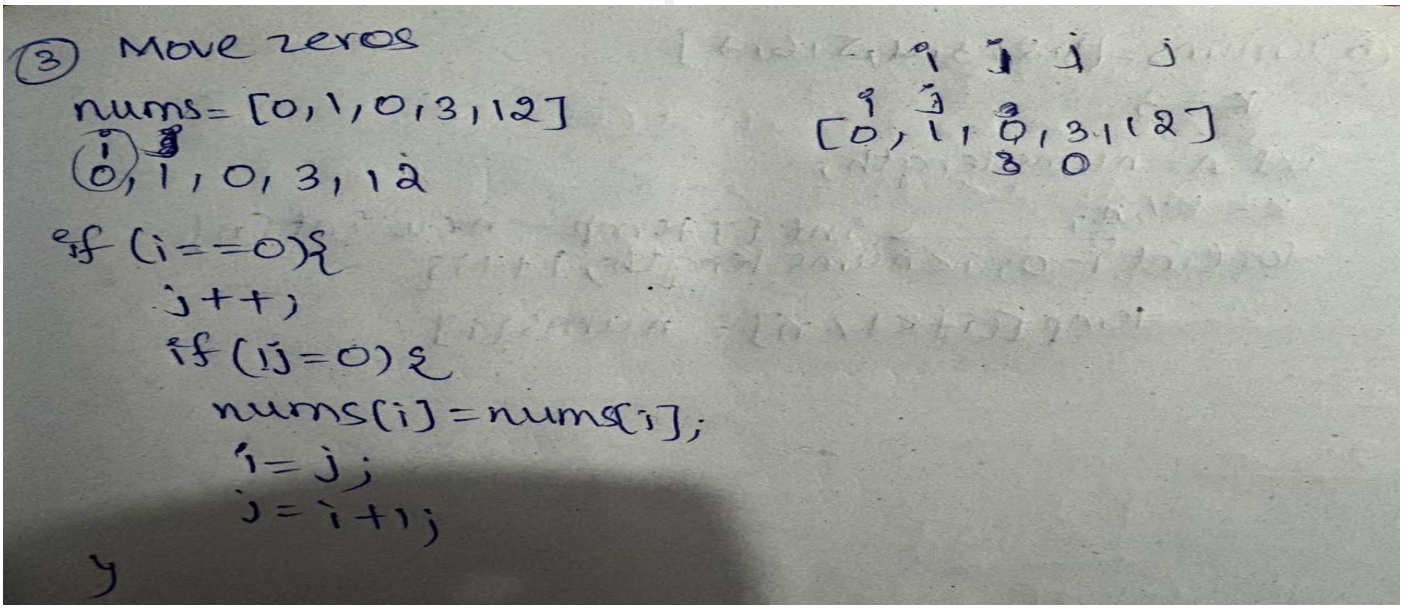
```java
class Solution {
    public void moveZeroes(int[] nums) {
        int i=0;
        int j=i+1;
        while(i<nums.length&&j<nums.length){
            if(nums[i] == 0){
                if(nums[j] != 0){
                    int temp = nums[i];
                    nums[i] = nums[j];
                    nums[j] = temp;
                    i++;
                    j = i+1;
                }
                else{
                    j++;
                }
            }
            else{
                i++;
                j = i+1;
            }
        }
        return ;
    }
}
```

Saved Ln 23, Col

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2



4.Maximum subarray LC-53

https://leetcode.com/problems/maximum-subarray/submissions/2186418827/

53. Maximum Subarray

Solved

Medium | Topics | Companies

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

```
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.
```

Example 2:

```
Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.
```

Example 3:

```
Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

Follow up: If you have figured out the `O(n)` solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

Java | Auto

```java
class Solution {
    public int maxSubArray(int[] nums) {

        int sum = nums[0];
        int max = nums[0];

        for(int i = 1; i < nums.length; i++) {

            sum = Math.max(nums[i], sum + nums[i]);

            max = Math.max(sum, max);
        }

        return max;
    }
}
```

Saved

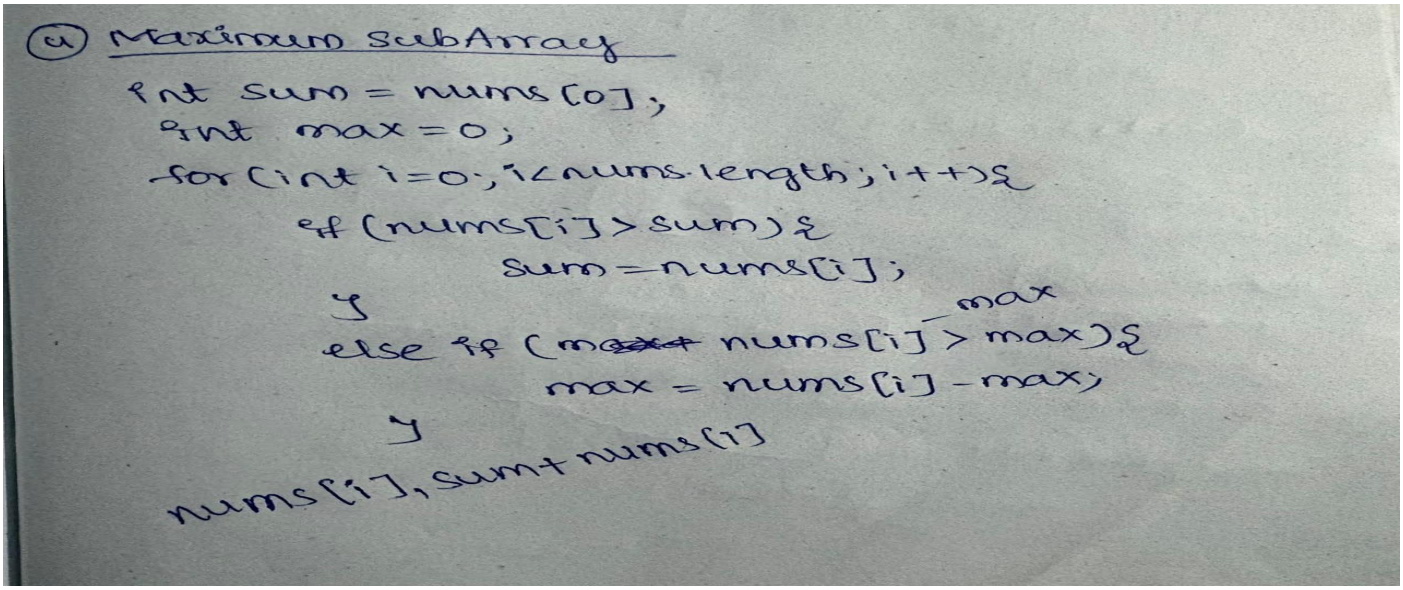
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[-2,1,-3,4,-1,2,1,-5,4]



5.Rotate Array LC-189

https://leetcode.com/problems/rotate-array/submissions/2186422155/

189. Rotate Array

Solved

Medium | Topics | Companies | Hint

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`
- `0 <= k <= 10^5`

Follow up:

21.7K | 764

Java | Auto

```java
class Solution {
    public void rotate(int[] nums, int k) {
    int n = nums.length;
    k = k%n;
    int [] temp = new int[n];
    for(int i=0;i<nums.length;i++){
        temp[(i+k) % n] = nums[i];
    }
    for(int i = 0; i < n; i++) {
            nums[i] = temp[i];
        }
    return ;
    }
}
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[1,2,3,4,5,6,7]

